WEEK 2 ｜完整參考筆記

C++ struct 與 class

從「把資料放在一起」到「設計安全、好用的自訂型別」

讀者設定	已經會變數、`if`、迴圈與函式；第一次系統學習自訂型別。
語言版本	主線範例使用 C++17；C++20 專屬語法會特別標示。
閱讀目標	看得懂、寫得出、能預測輸出，也能解釋為什麼這樣設計。

內容順序：重點 → 白話解釋 → 小例子 → 必要細節 → 練習與答案

目錄

先從這一頁開始 3

1 先建立物件與型別的心智模型 3

1.1 型別、物件、成員各自是什麼？ . . . 3

1.2 為什麼不用平行陣列就好？ . . . 4

2 struct：把相關資料組成一個型別 4

2.1 宣告、建立與點號存取 . . . 4

2.2 初始化：有給值、預設值、沒給值 . . . 5

2.3 巢狀 struct、陣列與 vector . . . 5

2.4 指標使用箭頭運算子 . . . 5

3 struct 與函式如何配合 6

3.1 傳值、傳參考、傳 const 參考 . . . 6

3.2 自訂型別也可以當回傳值 . . . 6

3.3 成員函式：讓操作靠近資料 . . . 7

4 struct 和 class 到底差在哪裡？ 7

5 class 與封裝：把規則守在型別裡 7

5.1 public、private、protected . . . 8

5.2 類別內宣告、類別外定義與:: . . . 8

5.3 getter 與 setter 不是越多越好 . . . 9

6 成員函式、const 與 this 9

6.1 函式後面的 const 保護誰？ . . . 9

6.2 const 多載：一般物件可改，const 物件只讀 . . . 9

6.3 this 指向目前的呼叫者 . . . 10

7 建構子與解構子：物件怎麼開始、怎麼結束 10

7.1 預設、參數化與多載建構子 . . . 10

7.2 成員初始化列表 . . . 11

7.3 delegating constructor：讓建構子重用建構子 . . . 11

7.4 explicit：避免單一參數被偷偷轉型 . . . 11

7.5 解構子與 RAII . . . 12

8 static、friend、巢狀型別與組合 12

8.1 static 成員屬於整個類別 . . . 12

8.2 friend 是精準授權，不是新的權限層級 . . . 13

8.3 組合：一個物件擁有另一個物件 . . . 13

9 複製、指定、移動與資源管理 13

9.1 先分清楚三個看似相近的動作 . . . 13

9.2 淺層複製為何可能出錯？ . . . 14

9.3 進階查閱：完整的深層複製骨架 . . . 14

9.4 move 的目的 . . . 15

10 運算子多載：讓自訂型別有一致的操作方式 15

10.1 能做什麼、不能做什麼 . . . 15

10.2 == 與!=：兩邊都不應被修改 . . . 15

10.3 += 修改左邊，+ 產生新結果 . . . 15

10.4 []：回傳參考才可以放在等號左邊 . . . 16

10.5 « 為何通常寫在類別外？ . . . 16

10.6 運算子設計檢查表 . . . 16

11 繼承與多型：class 的進階地圖 17

11.1 public inheritance 表示 is-a . . . 17

11.2 多型需要參考或指標 . . . 17

11.3 protected 與繼承權限 . . . 17

12 完整整合範例：安全的 BankAccount 18

13 常見錯誤：看錯誤訊息時先找哪裡？ 19

14 練習：從會讀到會設計 20

15 練習參考答案 21

16 一頁速查表 23

17 術語中英對照 24

先從這一頁開始

一句話抓住兩個主題

struct 與 class 都是在建立一種新的資料型別。這個型別可以同時描述「物件裡有什麼資料」和「物件可以做什麼事」。兩者最直接的語言差異，是未寫存取權限時，struct 預設 public，class 預設 private。

先看一個最小例子。輸入是學生的姓名與分數；型別把兩項資料綁成一組；輸出是完整學生資料。

```cpp
#include <iostream>
#include <string>
using namespace std;

struct Student {
    string name;
    int score;

    bool passed() const { return score >= 60; }
};

int main() {
    Student s{"Mina", 83};
    cout << s.name << ' ' << s.score << ' '
         << s.passed() << '\n';
}
```

執行順序是：

1. Student 宣告一種新型別，每個物件都有 name 與 score。
2. Student s{"Mina", 83}; 建立一個物件 s。
3. s.name 用點號讀取這個物件的姓名。
4. s.passed() 呼叫 s 自己的成員函式，檢查 s.score 是否至少 60。
5. 輸出是 Mina 83 1；最後的 1 代表 true。

建議閱讀路線

第一次讀：第 1、2、3、5、6、7、8 節，先略過標示「進階查閱」的段落。
準備考試：再讀第 4、9、10、11、12 節與速查表。
已會基礎：直接讀第 8 節的建構 / 解構、第 10 節複製語意、第 11 節運算子多載。

1 先建立物件與型別的心智模型

1.1 型別、物件、成員各自是什麼？

名稱	在本程式中的角色	具體例子
型別 type	規定資料長什麼樣、允許哪些操作	`Student`
物件 object	依照型別建立的一份實際資料	`Student s{"Mina", 83};` 中的 `s`
資料成員 data member	每個物件自己保存的欄位	`name`、`score`
成員函式 member function	由物件呼叫、可以使用該物件資料的函式	`passed()`
介面 interface	類別外部被允許使用的操作方式	公開的建構子、`deposit()`、`balance()`

小例子：同一型別可以建立很多物件

`Student a{"Amy", 90};` 與 `Student b{"Bo", 55};` 都依照 `Student` 建立，但兩者各自保存姓名與分數。修改 `a.score` 不會自動改到 `b.score`。

1.2 為什麼不用平行陣列就好？

若姓名和分數分開存，兩個容器的相同索引必須永遠代表同一人：

```
vector<string> names = {"Amy", "Bo"};
vector<int> scores = {90, 55};
```

一旦只刪到其中一邊，資料就錯配。改成 `vector<Student>` 後，每個學生的姓名與分數會一起移動：

```
vector<Student> students = {{"Amy", 90}, {"Bo", 55}};
```

2 struct：把相關資料組成一個型別

2.1 宣告、建立與點號存取

```
struct Point {
    int x;
    int y;
};  // 型別宣告結尾需要分號

Point p{3, 4};
cout << p.x << ' ' << p.y;  // 3 4
p.x = 10;                   // p 變成 (10, 4)
```

輸入 `{3, 4}` 依欄位宣告順序放入 `x`、`y`。`p.x` 中的點號表示「從物件 `p` 取得成員 `x`」。

最常見的第一個錯誤

`struct Point { ... }` 後面少了分號，編譯器常在下一行才報錯。看到「expected ';'」時，要回頭檢查型別定義結尾。

2.2 初始化：有給值、預設值、沒給值

```
struct Product {
    string name = "未命名";  // 預設成員初始值
    int price = 0;
};

Product a;                   // name="未命名", price=0
Product b{"筆", 25};         // 指定兩個欄位
Product c{};                 // 使用預設成員初始值
```

若內建型別欄位沒有預設值，`Product a;` 之類的寫法可能留下未初始化數值。對初學者最安全的習慣是：提供合理的預設成員初始值，或用大括號確實給值。

C++20 補充：指定欄位名稱

C++20 可以寫 `Point p{.x = 3, .y = 4};`。本講義主線使用 C++17，因此範例不依賴這種 designated initializer（指定初始化）語法。

2.3 巢狀 struct、陣列與 vector

```
struct Date {
    int year;
    int month;
    int day;
};

struct Event {
    string title;
    Date date;  // 一個 Event 內含一個 Date
};

Event exam{"期中考", {2026, 11, 5}};
cout << exam.date.month;  // 11

Student room[2] = {{"Amy", 90}, {"Bo", 55}};
vector<Student> group = {{"Cy", 72}, {"Di", 88}};
```

`exam.date.month` 是連續兩次成員存取：先從 `exam` 取得 `date`，再從那個 `Date` 取得 `month`。

2.4 指標使用箭頭運算子

```
Student s{"Mina", 83};
Student* ptr = &s;
```

```
cout << ptr->name;     // 等同 (*ptr).name
ptr->score = 90;       // 修改的是原本的 s.score
```

ptr 存的是 s 的位址。左邊是指標時使用 ->。例如 ptr->score 等同 (*ptr).score。

3 struct 與函式如何配合

3.1 傳值、傳參考、傳 const 參考

參數寫法	是否複製	函式能否改呼叫端原物件
Student s	會建立一份副本	不會；只改到副本
Student& s	不複製	會
const Student& s	不複製	不會；適合只讀

```
void addBonus(Student s) {       // 傳值：只改副本
    s.score += 5;
}

void addBonusInPlace(Student& s) { // 傳參考：改原物件
    s.score += 5;
}

void printStudent(const Student& s) { // 不複製，只讀
    cout << s.name << ' ' << s.score;
}
```

若 Student amy{"Amy", 80};：

- 呼叫 addBonus(amy) 後，amy.score 仍是 80。
- 呼叫 addBonusInPlace(amy) 後，amy.score 變成 85。
- printStudent(amy) 不複製，也不允許函式修改 amy。

3.2 自訂型別也可以當回傳值

```
Student better(const Student& a, const Student& b) {
    if (a.score >= b.score) return a;
    return b;
}
```

輸入是兩個唯讀的 Student；函式比較分數；輸出是一個新的 Student 值。同分時因為條件是 >=，會回傳第一位。

3.3 成員函式：讓操作靠近資料

```cpp
struct Rectangle {
    double width = 0;
    double height = 0;

    double area() const {
        return width * height;
    }
};

Rectangle r{3.0, 4.0};
cout << r.area();  // 12
```

area() 不需要另外接收 Rectangle 參數，因為它由 r 呼叫，能直接使用 r.width 與 r.height。函式尾端的 const 表示它承諾不修改呼叫它的物件。

4 struct 和 class 到底差在哪裡？

已由 C++ 語言直接規定的差異

- struct 的成員若未標示權限，預設是 public。
- class 的成員若未標示權限，預設是 private。
- 未寫繼承方式時，struct 預設公開繼承，class 預設私有繼承。

兩者都可以有：資料成員、成員函式、建構子、解構子、public/private/protected、運算子多載、繼承、樣板。所以下列說法是錯的：「struct 只能放資料，class 才能放函式。」

比較項目	常見 struct 用法	常見 class 用法
設計意圖	簡單資料組合，欄位可直接讀寫	維持規則，外部透過介面操作
預設成員權限	public	private
是否能有方法	可以	可以
是否能有建構子	可以	可以
編譯器本質	都能定義自訂型別	都能定義自訂型別

設計判斷

Point{x,y} 若只是交換資料，可以用 struct。銀行帳戶必須拒絕負數存款、拒絕超額提款，就適合用 class 隱藏餘額，再透過方法檢查規則。這是設計慣例，不是硬性規定。

5 class 與封裝：把規則守在型別裡

5.1 public、private、protected

```cpp
class BankAccount {
private:
    string owner;
    int balanceInDollars = 0;

public:
    bool deposit(int amount);
    bool withdraw(int amount);
    int balance() const;
};
```

- `public`：類別外可以使用，是提供給使用者的介面。
- `private`：只有類別自己的成員函式與被允許的 friend 能直接存取。
- `protected`：類別自己與衍生類別可直接存取；初學階段通常先少用。

封裝（*encapsulation*）是把資料和操作資料的函式放在同一型別，並限制外部只能透過公開介面改變狀態。它的直接效果是：輸入檢查只需集中在少數方法中。

```cpp
bool BankAccount::deposit(int amount) {
    if (amount <= 0) return false;
    balanceInDollars += amount;
    return true;
}

bool BankAccount::withdraw(int amount) {
    if (amount <= 0 || amount > balanceInDollars) return false;
    balanceInDollars -= amount;
    return true;
}

int BankAccount::balance() const {
    return balanceInDollars;
}
```

若餘額原本是 100：`withdraw(40)` 回傳 `true` 且餘額變 60；`withdraw(80)` 回傳 `false` 且餘額維持 60。

5.2 類別內宣告、類別外定義與::

`bool deposit(int amount);` 放在類別內，先告訴編譯器函式的名稱、輸入與輸出。實作前面的 `BankAccount::` 是 scope resolution operator（範圍解析運算子），表示這個 `deposit` 屬於 `BankAccount`。

```cpp
// BankAccount.h：通常放宣告
class BankAccount {
public:
    bool deposit(int amount);
```

```
};

// BankAccount.cpp：通常放定義
bool BankAccount::deposit(int amount) {
    // ...
}
```

小型練習可以全部放在一個 `.cpp`。專案變大後，分開宣告與定義可以縮短閱讀範圍，也讓多個來源檔共用同一介面。

5.3 getter 與 setter 不是越多越好

getter 是讀取方法，setter 是修改方法。若把每個 private 欄位都做成無條件 getter/setter，外部仍可能任意改壞狀態。

```
bool setScore(int newScore) {
    if (newScore < 0 || newScore > 100) return false;
    score = newScore;
    return true;
}
```

這個 setter 的輸入是新分數；先檢查 0 到 100；合法才改欄位並回傳 `true`；非法時回傳 `false`，而且保留原值。

6 成員函式、const 與 this

6.1 函式後面的 const 保護誰？

```
class Counter {
private:
    int value_ = 0;

public:
    void add(int amount) { value_ += amount; }
    int value() const { return value_; }
};
```

`add` 會修改呼叫者，所以不能加尾端 `const`。`value()` 只讀取資料，所以加上尾端 `const`。這讓 `const Counter fixed;` 可以呼叫 `fixed.value()`，卻不能呼叫 `fixed.add(1)`。

三個 const 的位置不要混在一起

`const Student& s`：參數 `s` 不能被修改。
`int score() const`：成員函式不能修改呼叫它的物件。
`const int* p`：不能透過指標 `p` 修改它所指的整數。

6.2 const 多載：一般物件可改，const 物件只讀

```cpp
class Pair {
private:
    int data_[2] = {0, 0};

public:
    int& at(int index) { return data_[index]; }
    const int& at(int index) const { return data_[index]; }
};
```

一般物件呼叫第一個版本，所以 `p.at(0) = 9;` 能修改欄位。`const Pair` 只能呼叫第二個版本，回傳唯讀參考，因此不能放在指定運算的左邊。

6.3 this 指向目前的呼叫者

```cpp
class Counter {
private:
    int value_ = 0;

public:
    Counter& add(int amount) {
        this->value_ += amount;
        return *this;
    }
};
```

若執行 `a.add(2)`，`this` 指向 `a`，`*this` 就是 `a` 本身。回傳 `Counter&` 讓 `a.add(2).add(3)` 連續操作同一物件。平常寫 `value_` 就足夠；參數和欄位同名時，`this->value_` 才特別有助於區分。

7 建構子與解構子：物件怎麼開始、怎麼結束

7.1 預設、參數化與多載建構子

```cpp
class Student {
private:
    string name_;
    int score_ = 0;

public:
    Student() : name_("未命名") {}

    Student(string name) : name_(name) {}

    Student(string name, int score)
        : name_(name) {
        setScore(score);
    }

    bool setScore(int score) {
        if (score < 0 || score > 100) return false;
```

```
        score_ = score;
        return true;
    }
};
```

同名但參數不同的建構子是 overloading（多載）。編譯器會依呼叫時提供的參數選版本：

```
Student a;              // 預設建構子
Student b("Amy");       // 一個 string
Student c("Bo", 85);    // string 加 int
```

7.2 成員初始化列表

```
class Point {
    const int id_;
    int x_;
    int y_;

public:
    Point(int id, int x, int y)
        : id_(id), x_(x), y_(y) {}
};
```

冒號後面是 member initializer list（成員初始化列表）。它在成員建立時直接給初值；建構子主體中的指定，是成員建立後才重新指定。`const` 成員、參考成員，以及沒有預設建構子的物件成員，都必須透過初始化列表處理。

初始化順序看宣告，不看列表順序

成員實際初始化順序是它們在類別內的宣告順序。即使寫 `: y_(...), x_(...)`，仍會先初始化先宣告的 `x_`。為避免誤會，初始化列表最好也照宣告順序寫。

7.3 delegating constructor：讓建構子重用建構子

```
class Point {
    int x_;
    int y_;

public:
    Point() : Point(0, 0) {}
    Point(int x, int y) : x_(x), y_(y) {}
};
```

`Point()` 把工作交給 `Point(int,int)`。這比兩個建構子各寫一次初始化規則更不容易不一致。

7.4 explicit：避免單一參數被偷偷轉型

```
class Meter {
```

```
    double value_;
public:
    explicit Meter(double value) : value_(value) {}
};

Meter a(3.5);       // 可以
// Meter b = 3.5;   // explicit 會阻止隱式轉換
```

單一參數建構子可能被編譯器當成「從參數型別轉成此類別」的方法。若這不是你要的行為，就加 explicit，要求呼叫端清楚寫出型別。

7.5 解構子與 RAII

```
class Trace {
    string name_;
public:
    explicit Trace(string name) : name_(name) {
        cout << "建立 " << name_ << '\n';
    }

    ~Trace() {
        cout << "離開 " << name_ << '\n';
    }
};
```

解構子與類別同名、前面有 ~、沒有參數，也沒有回傳型別。物件離開生命週期時自動執行。RAII 是「資源取得即初始化」：讓資源由物件保存，建構時取得，解構時自動釋放。實務上優先使用 string、vector、智慧指標等已經管理資源的型別。

不要手動呼叫一般區域物件的解構子

區域物件離開區塊時會自動解構。手動呼叫後，系統之後又自動呼叫，可能重複釋放資源。

8 static、friend、巢狀型別與組合

8.1 static 成員屬於整個類別

```
class Ticket {
private:
    int id_;
    static int nextId_;

public:
    Ticket() : id_(nextId_++) {}
    int id() const { return id_; }
    static int nextId() { return nextId_; }
};

int Ticket::nextId_ = 1;  // C++17 也可改用 inline static
```

每個物件各有自己的 `id_`，但所有 `Ticket` 共用一份 `nextId_`。static 成員函式沒有特定呼叫者，因此沒有 `this`，也不能直接讀取某一個物件的非 static 欄位。

8.2 friend 是精準授權，不是新的權限層級

```
class Point {
    int x_;
    int y_;
public:
    Point(int x, int y) : x_(x), y_(y) {}
    friend ostream& operator<<(ostream& out, const Point& p);
};

ostream& operator<<(ostream& out, const Point& p) {
    return out << '(' << p.x_ << ", " << p.y_ << ')';
}
```

`operator<<` 不是成員函式，但因為被宣告成 friend，可以讀取 `Point` 的 private 成員。friend 會增加耦合，只有外部函式確實需要看內部資料時才使用。

8.3 組合：一個物件擁有另一個物件

```
class Engine {
public:
    void start() { cout << "engine on\n"; }
};

class Car {
private:
    Engine engine_;  // Car 擁有一個 Engine
public:
    void start() { engine_.start(); }
};
```

composition（組合）表示「has-a」：`Car` 有一個 `Engine`。`Car` 建立時，`engine_` 先建立；`Car` 解構時，成員會自動解構。能用組合表達時，通常比為了重用幾行程式而繼承更清楚。

9 複製、指定、移動與資源管理

9.1 先分清楚三個看似相近的動作

```
Point a(1, 2);
Point b = a;   // 建立 b：copy constructor（複製建構子）
Point c(0, 0);
c = a;         // c 已存在：copy assignment（複製指定）
```

對只有 int、string、vector 等正常成員的類別，編譯器產生的複製通常已經正確。這叫 Rule of Zero：若成員自己會管理資源，就讓編譯器合成特殊成員函式，不必手寫。

9.2 淺層複製為何可能出錯？

```
class BadBuffer {
    int* data_;
public:
    explicit BadBuffer(int value) : data_(new int(value)) {}
    ~BadBuffer() { delete data_; }
};
```

若直接 BadBuffer b = a;，編譯器預設只複製指標位址，a.data_ 與 b.data_ 指向同一格記憶體。兩個物件解構時會對同一位址 delete 兩次，行為未定義。

Rule of Three、Five、Zero

- 若類別必須自己寫解構子、複製建構子、複製指定，通常三者都要一起思考：Rule of Three。
- C++11 後再加移動建構子與移動指定：Rule of Five。
- 最推薦的方向是讓 vector、string、unique_ptr 等成員管理資源，讓類別不必手寫這五個函式：Rule of Zero。

9.3 進階查閱：完整的深層複製骨架

```
class Buffer {
    size_t size_ = 0;
    int* data_ = nullptr;

public:
    explicit Buffer(size_t size)
        : size_(size), data_(new int[size]{}) {}

    ~Buffer() { delete[] data_; }

    Buffer(const Buffer& other)
        : size_(other.size_), data_(new int[other.size_]) {
        copy(other.data_, other.data_ + size_, data_);
    }

    Buffer& operator=(const Buffer& other) {
        if (this == &other) return *this;
        Buffer temp(other);
        swap(size_, temp.size_);
        swap(data_, temp.data_);
        return *this;
    }
};
```

複製建構子配置一塊新陣列，再複製每個元素，所以兩個物件不共用同一塊陣列。指定運算先檢查自我指定，再用 copy-and-swap（先做安全副本，再交換）維持例外安全。真正專案通常直接使用 `vector<int>`，把這些工作交給標準函式庫。

9.4 move 的目的

move semantics（移動語意）讓新物件接手暫時物件的資源，不必逐項複製大量資料。參數型別常寫成 `T&&`。初學者先掌握兩點即可：

- 標準容器已正確實作 move；使用 `vector` 通常不必自己處理。
- 被移動後的物件仍可安全解構、可以再指定，但原內容通常不能再假設。

10 運算子多載：讓自訂型別有一致的操作方式

10.1 能做什麼、不能做什麼

operator overloading（運算子多載）能替自訂型別定義現有運算子的行為，例如 `==`、`+`、`[]`、`<<`。它不能創造新符號、不能改變優先順序、不能改變運算元個數，也至少要有一個運算元是自訂型別。

10.2 == 與!=：兩邊都不應被修改

```
bool operator==(const Point& other) const {
    return x_ == other.x_ && y_ == other.y_;
}

bool operator!=(const Point& other) const {
    return !(*this == other);
}
```

`a == b` 可看成 `a.operator==(b)`。參數的 `const&` 保護右邊並避免複製；尾端 `const` 保護左邊。

10.3 += 修改左邊，+ 產生新結果

```
Point& operator+=(const Point& other) {
    x_ += other.x_;
    y_ += other.y_;
    return *this;
}

Point operator+(const Point& other) const {
    Point result(*this);
    result += other;
    return result;
}
```

`a += b` 會改 `a`，所以回傳 `Point&` 指向原本左邊。`a + b` 不改兩邊，先複製左邊到 `result`，再重用

+=，最後回傳新的值。不能回傳 Point& 指向區域變數 result，因為函式結束後它就不存在。

10.4 []：回傳參考才可以放在等號左邊

```cpp
int& operator[](size_t index) {
    if (index == 0) return x_;
    if (index == 1) return y_;
    throw out_of_range("Point index must be 0 or 1");
}

const int& operator[](size_t index) const {
    if (index == 0) return x_;
    if (index == 1) return y_;
    throw out_of_range("Point index must be 0 or 1");
}
```

第一個版本讓 p[0] = 9; 修改 x_。第二個版本讓 const Point 可以讀，卻不能寫。非法索引丟出例外，而不是讀取不存在的記憶體。

10.5 « 為何通常寫在類別外？

```cpp
ostream& operator<<(ostream& out, const Point& p) {
    out << '(' << p.x() << ", " << p.y() << ')';
    return out;
}
```

cout << p 的左邊是 ostream，不是 Point，所以不能做成 Point 的成員函式。回傳 ostream& 才能繼續串接下一個輸出。如果公開 getter 足夠，這個函式不必是 friend。

10.6 運算子設計檢查表

運算子	是否改左邊	常見回傳型別	常見 const 設計
==, !=	否	bool	參數 const&，方法 const
+	否	T	參數 const&，方法 const
+=	是	T&	參數 const&，方法不可 const
[] 讀取	否	const T& 或 T	方法 const
[] 寫入	可能	T&	方法不可 const
<<	否	ostream&	物件參數 const&

11 繼承與多型：class 的進階地圖

這節的定位

繼承與執行期多型也是 class 能力，但通常應在學生熟悉封裝、參考與生命週期後再深入。這裡提供完整地圖與安全骨架，不把它混進第一輪主線。

11.1 public inheritance 表示 is-a

```
class Shape {
public:
    virtual double area() const = 0;
    virtual ~Shape() = default;
};

class Circle : public Shape {
    double radius_;
public:
    explicit Circle(double radius) : radius_(radius) {}
    double area() const override {
        return 3.141592653589793 * radius_ * radius_;
    }
};
```

`Circle : public Shape` 表示 Circle 是一種 Shape。`virtual` 讓經由基底類別參考或指標呼叫時，執行實際物件的版本。`= 0` 是純虛擬函式，讓 Shape 成為抽象類別，不能直接建立物件。`override` 要求編譯器檢查是否真的覆寫成功。

11.2 多型需要參考或指標

```
void printArea(const Shape& shape) {
    cout << shape.area() << '\n';
}

Circle c(2.0);
printArea(c);  // 呼叫 Circle::area()
```

若用值把衍生物件存成基底物件，衍生部分會被切掉，稱為 object slicing（物件切片）。需要多型時使用基底參考或智慧指標。

11.3 protected 與繼承權限

`protected` 成員可由衍生類別直接存取，但外部不行。通常仍優先把資料設為 private，透過 protected/public 函式維持規則。public inheritance 會保留基底的 public 介面；private inheritance 比較少用，含義也不是一般的 is-a。

12 完整整合範例：安全的 BankAccount

```cpp
#include <iostream>
#include <string>
using namespace std;

class BankAccount {
private:
    string owner_;
    int balance_ = 0;
    static int accountCount_;

public:
    explicit BankAccount(string owner)
        : owner_(owner) {
        ++accountCount_;
    }

    BankAccount(string owner, int initialBalance)
        : BankAccount(owner) {
        deposit(initialBalance);
    }

    ~BankAccount() { --accountCount_; }

    const string& owner() const { return owner_; }
    int balance() const { return balance_; }
    static int accountCount() { return accountCount_; }

    bool deposit(int amount) {
        if (amount <= 0) return false;
        balance_ += amount;
        return true;
    }

    bool withdraw(int amount) {
        if (amount <= 0 || amount > balance_) return false;
        balance_ -= amount;
        return true;
    }

    bool transferTo(BankAccount& target, int amount) {
        if (&target == this) return false;
        if (!withdraw(amount)) return false;
        target.balance_ += amount;
        return true;
    }
};

int BankAccount::accountCount_ = 0;

```

```cpp
int main() {
    BankAccount amy("Amy", 100);
    BankAccount bo("Bo", 20);

    cout << amy.transferTo(bo, 40) << '\n';
    cout << amy.balance() << ' ' << bo.balance() << '\n';
    cout << BankAccount::accountCount() << '\n';
}
```

預期輸出：

```
1
60 60
2
```

因果流程：`transferTo` 先拒絕自己轉給自己；再呼叫 `withdraw` 檢查金額與餘額；提款成功後才增加目標帳戶餘額。成員函式可以讀寫同類別另一個物件的 private 欄位，所以 `target.balance_` 合法。

13 常見錯誤：看錯誤訊息時先找哪裡？

症狀	常見原因	修正方向
型別定義下一行報錯	class/struct 結尾少分號	檢查右大括號後是否有 `;`
private within this context	類別外直接存取 private	改用公開方法，或重新設計介面
no matching constructor	傳入參數與建構子不相符	核對數量、型別與 explicit
deleted default constructor / no default constructor	已宣告其他建構子後，編譯器未必提供無參數版本	明確寫 `T() = default;` 或提供預設建構子
discard qualifiers / const 錯誤	const 物件呼叫了非 const 方法	若方法只讀資料，補尾端 `const`
multiple definition	在標頭重複定義非 inline 的 static 資料或函式	宣告、定義分開；或 C++17 用 `inline static`
程式偶爾出現怪值	內建型別欄位未初始化	給預設成員初始值或建構子初始化
double free / crash	擁有裸指標卻只做淺層複製	優先改用 vector/智慧指標，或正確實作 Rule of Five
結果改不到原物件	參數用傳值，改到副本	若確實要改呼叫端，使用非 const 參考
回傳後資料壞掉	回傳區域變數的參考或指標	區域結果應回傳值

14 練習：從會讀到會設計

練習 1 | struct 與函式

宣告 Book，包含書名、頁數與價格。寫 printBook(const Book&)，以及 cheaper(const Book&, const Book&)，同價時回傳第一本。

練習 2 | 封裝與邊界

設計 Temperature 類別，內部保存攝氏溫度。建構子收到低於絕對零度 -273.15 的輸入時，改存 -273.15。提供 celsius() 與 fahrenheit()；兩個都不可修改物件。

練習 3 | 建構子與 setter

設計 class Student。分數必須是 0 到 100；非法建構值保留 0；setScore 失敗時保留原值並回傳 false。

練習 4 | const 與參考

解釋下列三個函式的差異，並寫出每個函式是否會複製、是否能改呼叫端：

```
void f(Student s);
void g(Student& s);
void h(const Student& s);
```

練習 5 | 運算子多載

替 Vec2 實作 ==、+=、+、兩個版本的 [] 與輸出 <<。測試 a += a、a + b + c、const Vec2 讀索引、非法索引。

練習 6 | 獨立設計題

設計 class InventoryItem：名稱、單價、庫存量是 private。補貨數量必須為正；出售數量必須為正且不可超過庫存；總價以「單價乘庫存」計算。列出至少 6 個邊界測試。

15 練習參考答案

答案 1

```cpp
struct Book {
    string title;
    int pages;
    int price;
};

void printBook(const Book& book) {
    cout << book.title << ' ' << book.pages << ' ' << book.price;
}

Book cheaper(const Book& a, const Book& b) {
    return (a.price <= b.price) ? a : b;
}
```

答案 2

```cpp
class Temperature {
    double celsius_;
public:
    explicit Temperature(double celsius)
        : celsius_(celsius < -273.15 ? -273.15 : celsius) {}

    double celsius() const { return celsius_; }
    double fahrenheit() const { return celsius_ * 9.0 / 5.0 + 32.0; }
};
```

答案 3

```cpp
class Student {
    string name_;
    int score_ = 0;
public:
    Student(string name, int score) : name_(name) {
        setScore(score);
    }

    bool setScore(int score) {
        if (score < 0 || score > 100) return false;
        score_ = score;
        return true;
    }

    int score() const { return score_; }
};
```

答案 4

f 傳值，會複製，不能改呼叫端原物件；g 傳非 const 參考，不複製而且可以改原物件；h 傳 const 參考，不複製也不能改原物件。只讀取大型物件時，h 通常最合適。

答案 5：核心類別

```
class Vec2 {
    int x_;
    int y_;
public:
    Vec2(int x, int y) : x_(x), y_(y) {}

    int x() const { return x_; }
    int y() const { return y_; }

    bool operator==(const Vec2& other) const {
        return x_ == other.x_ && y_ == other.y_;
    }

    Vec2& operator+=(const Vec2& other) {
        x_ += other.x_;
        y_ += other.y_;
        return *this;
    }

    Vec2 operator+(const Vec2& other) const {
        Vec2 result(*this);
        result += other;
        return result;
    }

    int& operator[](size_t index) {
        if (index == 0) return x_;
        if (index == 1) return y_;
        throw out_of_range("Vec2 index must be 0 or 1");
    }

    const int& operator[](size_t index) const {
        if (index == 0) return x_;
        if (index == 1) return y_;
        throw out_of_range("Vec2 index must be 0 or 1");
    }
};

ostream& operator<<(ostream& out, const Vec2& v) {
    return out << '(' << v.x() << ", " << v.y() << ')';
}
```

答案 6：測試重點

至少測：正常補貨、補貨 0、負數補貨、正常出售、出售 0、負數出售、出售剛好等於庫存、出售超過庫存、庫存變成 0 後再出售。每個失敗操作都要確認庫存沒有被改動。

16 一頁速查表

基本骨架

```
struct Data {
    int value = 0;       // 預設 public
};

class SafeType {
private:
    int value_ = 0;
public:
    explicit SafeType(int value) : value_(value) {}
    int value() const { return value_; }
    bool setValue(int value);
};
```

選參數方式

- 小型內建值且不需改原值：傳值，例如 `int amount`。
- 要修改呼叫端：`T&`。
- 只讀且避免複製：`const T&`。

選回傳方式

- 產生新結果：回傳 `T`。
- 回傳已存在且生命週期足夠長的物件：才考慮 `T&` 或 `const T&`。
- 永遠不要回傳區域變數的參考或指標。

設計 class 前的五個問題

1. 每個物件要保存哪些資料？初值是什麼？
2. 哪些狀態不合法？誰負責阻止它？
3. 外部真正需要哪些 public 操作？
4. 哪些方法只讀，應該標成 `const`？
5. 類別是否直接擁有資源？能否改用標準容器以遵守 Rule of Zero？

17 術語中英對照

中文	英文	在本筆記中的意思
自訂型別	user-defined type	程式設計者用 struct/class 建立的型別
物件	object	依型別建立的一份實際資料
實例	instance	物件的另一種常見稱呼
資料成員	data member	物件保存的欄位
成員函式	member function	由物件呼叫、可存取其成員的函式
存取權限	access control	public/private/protected 的限制
封裝	encapsulation	把資料與操作放在一起並限制外部存取
介面	interface	類別外可用的公開操作
建構子	constructor	物件建立時執行的特殊函式
解構子	destructor	物件生命週期結束時執行的特殊函式
成員初始化列表	member initializer list	在建構子主體前直接初始化成員
函式多載	function overloading	同名、參數不同的多個函式
運算子多載	operator overloading	定義運算子用於自訂型別時的行為
組合	composition	一個物件擁有另一個物件
繼承	inheritance	衍生類別沿用並擴充基底類別介面
多型	polymorphism	透過共同介面呼叫不同實際型別的行為
淺層複製	shallow copy	只複製位址等表面成員，可能共用資源
深層複製	deep copy	建立獨立資源並複製內容
移動語意	move semantics	把資源所有權轉交給新物件

最後自我檢查

確認你能回答：`struct`/`class` 的預設權限有何差別？`private`、初始化列表與尾端 `const` 各解決什麼問題？何時使用 `T`、`T&`、`const T&`？為什麼 `+` 回傳值、`+=` 回傳參考？裸指標的預設複製有何風險？